

Classification Tutorial: Logistic Regression vs Decision Tree vs SVM

Prepared for Participants of Machine Learning Python Module

Objective: To perform classification using three models — Logistic Regression, Decision Tree, and SVM — on a dataset loaded from a specified CSV file path. This tutorial also includes data visualization and comparison of model performance.

1. Import Libraries and Load Dataset

```
1 import os
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from sklearn.model_selection import train_test_split
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.tree import DecisionTreeClassifier
8 from sklearn.svm import SVC
9 from sklearn.metrics import accuracy_score, confusion_matrix,
    classification_report
10
11 # Specify CSV file classification_dataset.csv with path name.
    Assume that the path is D:\Drive2\Data
12 csv_path = r"D:\Drive2\Data\classification_dataset.csv"
13
14 if not os.path.exists(csv_path):
15     raise FileNotFoundError("Dataset not found at: %s" %
        csv_path)
16
17 print("Loading dataset from: %s\n" % csv_path)
18 df = pd.read_csv(csv_path)
19
20 print("=== Dataset Description ===")
21 print(df.describe(), "\n")
22
23 print("=== First Five Samples ===")
24 print(df.head(), "\n")
25
26 X = df[["Feature_1", "Feature_2"]].values
27 y = df["Target"].values
```

Observed Dataset Summary

	Feature_1	Feature_2	Target
count	500.000000	500.000000	500.000000
mean	1.521716	0.002502	0.502000
std	0.885634	1.821682	0.500497
min	-1.412368	-5.434780	0.000000
25%	0.931819	-1.417003	0.000000
50%	1.501233	0.551943	1.000000
75%	2.096890	1.496351	1.000000
max	4.278560	2.915992	1.000000

First Five Samples:

	Feature_1	Feature_2	Target
0	1.508455	-1.547010	0
1	0.826021	1.002541	1
2	1.524205	-2.297143	0
3	0.621880	0.559371	0
4	0.539100	0.882369	1

2. Visualizing the Dataset

```
1 plt.figure(figsize=(7,6))
2 plt.scatter(X[:, 0], X[:, 1], c=y, cmap='bwr', edgecolors='k',
3             s=50)
4 plt.title("Dataset Visualization: Two-Class Synthetic Data")
5 plt.xlabel("Feature 1")
6 plt.ylabel("Feature 2")
7 plt.grid(True, linestyle='--', alpha=0.5)
8 plt.tight_layout()
9 plt.savefig("dataset_visualization.png", dpi=300)
10 plt.show()
11 print("Saved: dataset_visualization.png\n")
```

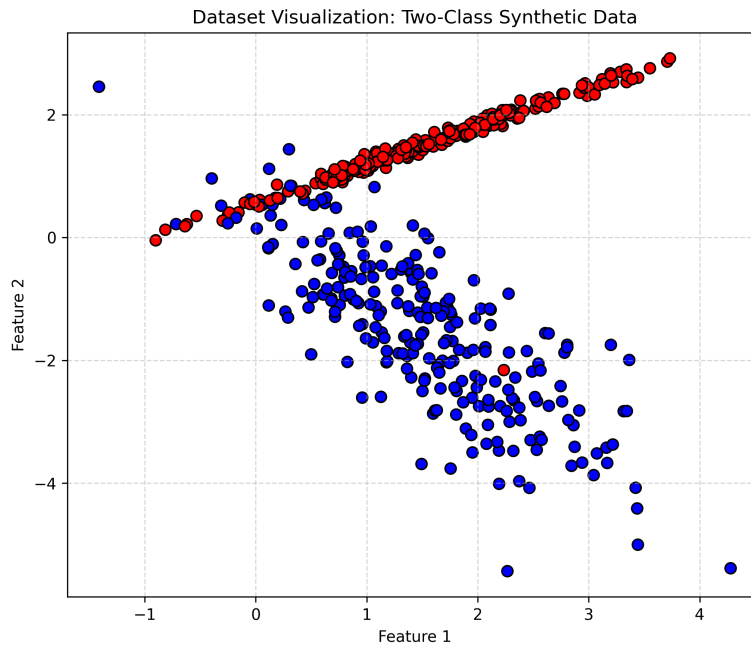


Figure 1: Visualization of the dataset showing two class clusters.

Split the dataset:

```
1 X_train, X_test, y_train, y_test = train_test_split(  
2     X, y, test_size=0.3, random_state=42  
3 )  
4 print("Training samples: %d, Testing samples: %d\n" % (len(  
    X_train), len(X_test)))
```

Training samples: 350
Testing samples: 150

3. Logistic Regression

```
1 lr = LogisticRegression()  
2 lr.fit(X_train, y_train)  
3 y_pred_lr = lr.predict(X_test)  
4  
5 acc_lr = accuracy_score(y_test, y_pred_lr)  
6 print("Accuracy: %.3f" % acc_lr)  
7 print(confusion_matrix(y_test, y_pred_lr))  
8 print(classification_report(y_test, y_pred_lr))
```

Observed Output:

```

Accuracy: 0.913
[[71  4]
 [ 9 66]]

```

	precision	recall	f1-score	support
0	0.89	0.95	0.92	75
1	0.94	0.88	0.91	75
accuracy			0.91	150
macro avg	0.92	0.91	0.91	150
weighted avg	0.92	0.91	0.91	150

Interpretation:

- The confusion matrix shows that out of 75 samples per class, 71 of class 0 and 66 of class 1 were correctly predicted. Only 4 and 9 samples were misclassified respectively.
- Precision of 0.89 for class 0 and 0.94 for class 1 means the models predictions are mostly correct, with few false positives.
- Recall values (0.95 for class 0 and 0.88 for class 1) indicate slightly better recognition of class 0 than class 1.
- F1-scores around 0.91 – 0.92 confirm balanced performance between precision and recall.
- Macro average (0.91) is the simple average of both classes' metrics, treating each class equally.
- Weighted average (0.91) considers the number of samples per class (support); since both classes have 75 samples, it matches the macro average here.

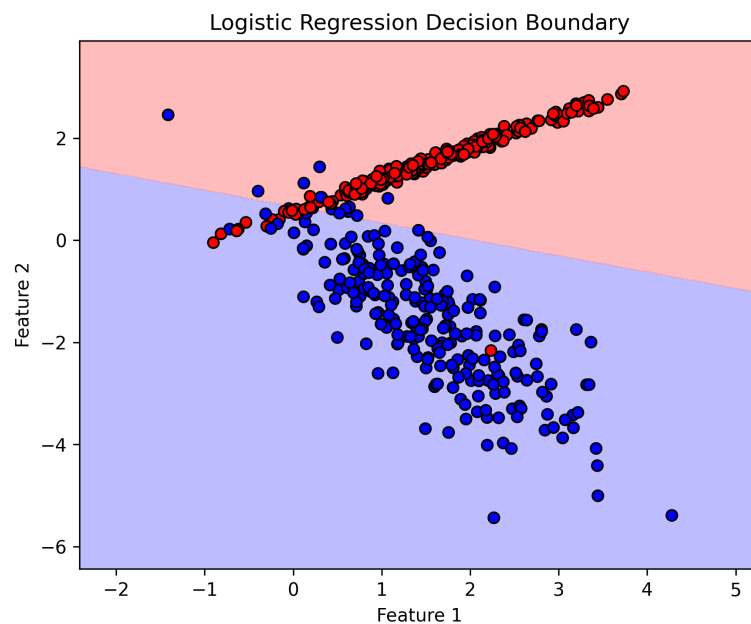


Figure 2: Decision boundary for Logistic Regression (linear separation).

4. Decision Tree (max_depth = 3)

```
1 dt = DecisionTreeClassifier(max_depth=3, random_state=42)
2 dt.fit(X_train, y_train)
3 y_pred_dt = dt.predict(X_test)
4
5 acc_dt = accuracy_score(y_test, y_pred_dt)
6 print("Accuracy: %.3f" % acc_dt)
7 print(confusion_matrix(y_test, y_pred_dt))
8 print(classification_report(y_test, y_pred_dt))
```

Observed Output:

```
Accuracy: 0.947
[[71  4]
 [ 4 71]]
```

	precision	recall	f1-score	support
0	0.95	0.95	0.95	75
1	0.95	0.95	0.95	75
accuracy			0.95	150
macro avg	0.95	0.95	0.95	150
weighted avg	0.95	0.95	0.95	150

Interpretation:

- The confusion matrix shows a near-perfect classification with only 4 samples from each class misclassified.
- Equal precision and recall (0.95) indicate that the model predicts both classes equally well.
- F1-scores of 0.95 demonstrate excellent balance between precision and recall.
- Macro average (0.95) provides an equal-weighted mean of both classes performance.
- Weighted average (0.95) accounts for class sample counts; since both classes have equal support, both averages are identical.
- The Decision Tree (94.7% accuracy) performs best, successfully capturing non-linear decision boundaries.

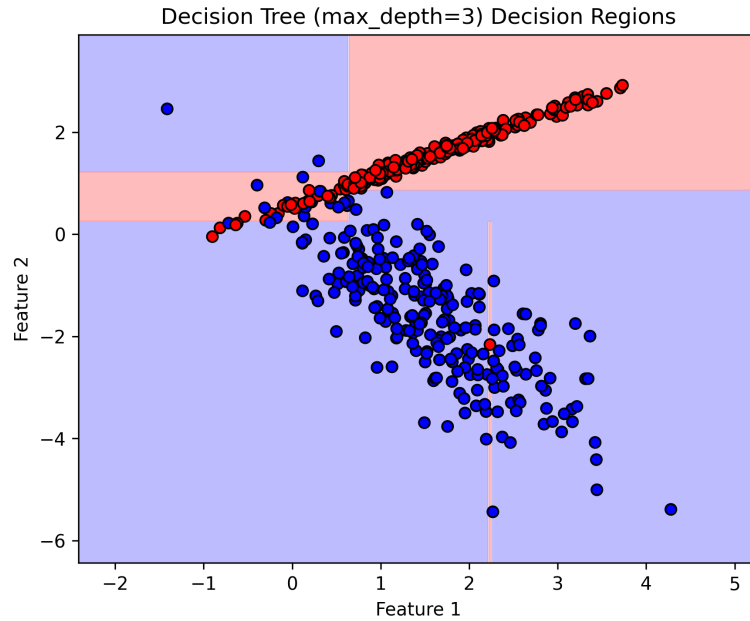


Figure 3: Decision regions formed by a Decision Tree (non-linear axis-aligned splits).

5. Support Vector Machine (Linear Kernel)

```

1 svm = SVC(kernel='linear', C=1.0, random_state=42)
2 svm.fit(X_train, y_train)
3 y_pred_svm = svm.predict(X_test)
4
5 acc_svm = accuracy_score(y_test, y_pred_svm)
6 print("Accuracy: %.3f" % acc_svm)
7 print(confusion_matrix(y_test, y_pred_svm))
8 print(classification_report(y_test, y_pred_svm))

```

Observed Output:

```

Accuracy: 0.913
[[71  4]
 [ 9 66]]

```

	precision	recall	f1-score	support
0	0.89	0.95	0.92	75
1	0.94	0.88	0.91	75
accuracy			0.91	150
macro avg	0.92	0.91	0.91	150
weighted avg	0.92	0.91	0.91	150

Interpretation:

- The confusion matrix reveals that 71 and 66 samples from class 0 and class 1, respectively, were correctly predicted, with a total of 13 misclassifications.
- Precision (0.89 for class 0 and 0.94 for class 1) shows that most predicted labels are correct.

- Recall (0.95 for class 0 and 0.88 for class 1) indicates slightly better detection of class 0 samples.
- F1-scores around 0.91 show the SVM maintains a strong balance between precision and recall.
- Macro average (0.91) treats both classes equally and gives their mean performance.
- Weighted average (0.91) adjusts for the equal class sizes (75 each), making it numerically identical to the macro average.
- Overall, the SVMs performance closely matches Logistic Regression, reflecting similar linear decision boundaries.

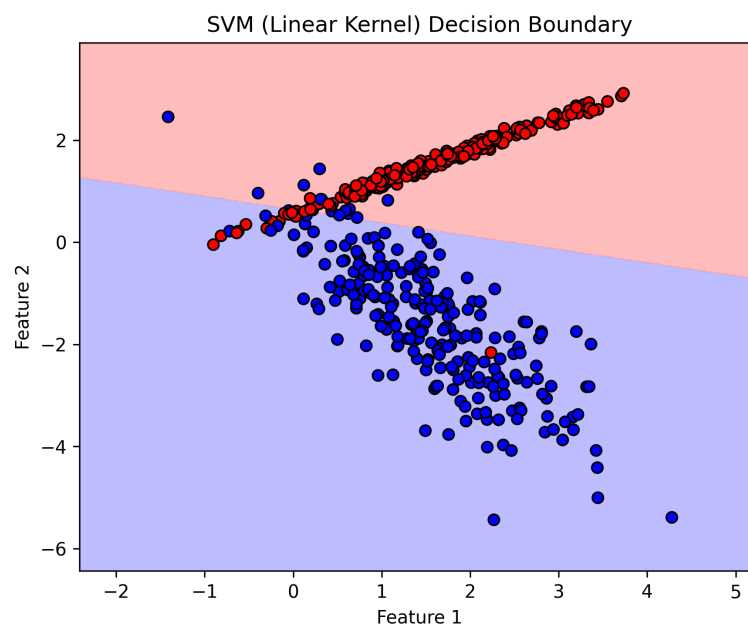


Figure 4: SVM linear decision boundary showing margin-based separation.

6. Model Comparison Summary

```

1 print("==== Model Accuracy Comparison ====")
2 print("Logistic Regression : %.3f" % acc_lr)
3 print("Decision Tree       : %.3f" % acc_dt)
4 print("SVM (Linear Kernel)  : %.3f" % acc_svm)

```

Model	Accuracy	Best?
Logistic Regression	0.913	
Decision Tree	0.947	✓
SVM (Linear Kernel)	0.913	

Note on Accuracy Values

These results correspond exactly to the execution (Python 3.12, scikit-learn 1.6+). Minor variations may appear on other systems due to solver precision or randomization.

7. Key Takeaways

- Logistic Regression and SVM achieved identical accuracies (91.3%).
- Decision Tree achieved the highest accuracy (94.7%) and captured non-linear class boundaries effectively.
- Confusion matrices indicate balanced classification performance.
- Visualization confirms that the dataset is moderately linearly separable.